Solving the LunarLanderContinuous-v2

Itai Shalik (ID. 206763690),Ariel Maymon (ID. 208934216)

Submitted as final project report for the RL course, BIU, 2021

1 Introduction

Over the past decade, reinforcement learning has proven significant contribution in variety of applications, such as Robotics, Games and Autonomous Driving. The breakthroughs in all these areas were made possible due to advanced algorithms and methods developed in recent years (especially at DeepMind), such as DQN and its variants, Policy Gradient, Actor-Critic methods amongst others. In this paper, we solve the lunar lander problem - a well known problem from OpenAI gym using multiple deep reinforcement learning algorithms, compare between the different approaches, and at last testing the models adaptation by adding zero-mean Gaussian noise into the environment. The environment is defined as solved once an agent scores an average of 200 points over 100 episodes. Both the observation and action space are continuous, where the observation space is 8 dimensions and the action space is two dimensional. This problem is interesting due to its resemblance to actual moon landing which is a difficult task on its own, the continuous state and action space, which face a challenge to "standard" reinforcement learning algorithms, and are one of the reasons of the advances in this area of research, which will therefore be demonstrated in this paper. Our focus in this paper is to explore the realm of reinforcement learning and gain a deeper understanding of the various techniques and approaches which are used nowadays both in academia and in practice.

1.1 Related Works

Although there are several implementations of different algorithms to solve the lunar lander problem, none is solely focused on solving it using deep learning methods. The authors of [12] solved the environment using SARSA and tabular Q-Learning, with a section devoted to added noise but it was the discrete environment of Lunar Lander and it didn't utilize more advanced methods. Besides that, most of the academic world usually tests their assumptions on different OpenAI environments than ours.

2 Solution

2.1 General approach

Since we do not have any prior knowledge of the environment, even intense effort of exploration can still possess some bias to the agent which may still lead to sub-optimal behavior. Therefore, choosing model-free methods seems as a reasonable approach to train the agents with.

However, not without any concerns: The continuous nature of the state and action spaces raise a difficulty for tabular methods, which will have to incorporate some kind of discretization or function approximations for these spaces to make them work. Even then, this can become infeasible to train if the spaces' dimensions are too large. Moreover, a different problem that may arise to some of the deep Q-Learning algorithms that are bootstrapped, off-policy and are using function approximations (such as Neural Nets) is the deadly triad, as mentioned in Sutton and Barto[8], and in more detail[11], which might cause instability or divergence of the models in the worst case. In general, as we come to solve this problem, there are two main approaches to represent a learning agent in a model-free environment: Policy Optimization and Q-Learning. The Policy Optimization family optimize (usually) on-policy, such that each update only uses current policy information, while Q-Learning methods are performed off-policy and update using any data collected during training phase.

Since Policy Optimization techniques directly optimize the policy (which we are looking to make optimal), they tend to be more reliable and stable to train, not as Q-Learning algorithms which optimize indirectly for it (by the Q-function), making it less stable. On the other hand, Q-Learning methods has the advantage of reusing data as Policy methods are somewhat bounded to sample from its newest temporary policy, making them more sample efficient.

Taking all of the above into consideration, we decided to tackle the environment using various approaches: Tabular Q-Learning [1], Deep Q-Networks (DQN)[2] and its improvements (such as prioritzed experience replay, Double DQN, Dueling DDQN), which are the Deep Learning framework of the vanilla Q-Learning algorithm.

In addition to that, we try to solve it also by using more recent, state-of-the-art models which are policy governed - the Proximal Policy Optimization (PPO)[7], which utilizes the clipped surrogate objective with multiple epochs of policy updates which makes it stable (in terms of not drifting too much from current policy) and more sample efficient than other policy optimization methods, and the Soft-Actor Critic[10] algorithm which optimizes a stochastic policy in a off-policy way, as its main contribution is the entropy regularization term which is added in order to maximize the "randomness" of the policy where it might get stuck in sub-optimal behavior. It is usually harder to implement than PPO, but is more sample efficient as we use an experience replay buffer.

The tabular method should provide us with the insight on if and how discretization may work, and if it's feasible for such an environment. The Deep Q-Networks we train are very much tailored for this environment and are con-

sidered the fundamental approach for this environment so we expect to see it behaves well. We will include a few common improvements to the Vanilla DQN algorithms to compare it and see they actually help to solve the Lunar Lander problem and to what degree, and also if it helps the inherent instability of this family of algorithms.

Lastly, the Policy Optimization techniques should provide an interesting comparison between both families of algorithms, maybe deriving use-cases for each one.

2.2 Design

We have decided to use the python language with PyTorch framework for their high prevalence in such applications.

We have trained a total of 6 different models. the first 4 models are DQN variants, which include both vanilla and Dueling neural networks[5] for the network architecture, and the simple versus the Double-DQN[4] update rule. Each of these models took approximately half an hour to train until model successfully solves the environment. We also tried both regular and prioritized experience replay, but couldn't get the models with PER to converge in a reasonable time, so we did not include them. We believe something in our implementation was missing.

The last two models are the PPO and the SAC algorithms. The PPO algorithm took X hours until convergence, whilst SAC took X hours.

The tabular Q-Learning algorithm failed to converge even after more than 10,000 episodes (although showing minimal improvement), which afterwards we didn't continue. we have tried to discretize the observation space multiple times, approximate it using linear function approximators, tuning the hyperparameters and the number of action space operations (as it is also discretized here) to no avail. It seems as the problem dimensions are too large for the algorithm to converge in reasonable time, although due to time constraints we couldn't address it any further. There might exist a better approximators or discretization that are able to drive this method to converge well, as [12] managed to get the discrete environment to converge.

The DQN family was able to converge, where tuning the parameters also increase the convergence rate. We found for the policy methods that improper selection of the parameters can sometimes cause what seems as divergence, especially for the PPO agent.

2.2.1 DQN

All DQN variants were trained using the same hyper-parameters to be able to compare their performance. let s be the state (observation) state dimension, and a the action state dimension. As DQN needs a discrete action size, we decided to discretize the action space into 4 action for the first dimension and 3 for the second, giving a total of 4*3 = 12 discrete actions. let a' denote the

number of discretized actions. The networks' structure is [s,64,64,a'], same of course goes for the target network.

2.2.2 SAC

For the Soft Actor Critic method, no discretization is needed. The policy is learnt by maximizing a trade-off between the reward and the entropy of the policy, which is somewhat connected to the exploration-exploitation trade-off we are familiar with from the DQN algorithm. We chose to implement a newer version of this algorithm, which omits the Value function and optimizes a varying a (the trade-off parameter). We have two Q-function networks, whose layers' dimensions are [$s + a$,64,64,1] (as it gives a q-value for each state-action pair), and a policy-function network whose dimensions are [s,64,64,a], which gives for each state the probability distribution over each action.

2.2.3 PPO

We used the PPO algorithm with the clipped objective cost function. We maintain two policy networks, the first one as the policy to optimize and the second for collecting our samples, which works similar to a target network on other algorithms. The ratio between the new and old policies may vary significantly between updates, which we would like to avoid as it may cause drastic gradient updates that reflects on our policy, we clip the variance by introducing a parameter ϵ that bounds the ratio gradient update later on.

We have tried two versions of the PPO algorithm: discrete and continuous. While we we able to get the discrete PPO the solve the environment by tweaking some of the parameters, the continuous approach took us too long to converge. In general, we notice fluctuations of the average reward in both versions.

3 Experimental results

We have run each of the algorithms until convergence, with 3 iterations with different seeds for every agent. As Fig 1 shows, DQN multiple agents took about 900-1000 episodes on average to converge, with the SAC taking around 350 episodes. Our PPO implementation didn't keep up, as the runs that did converge to solution took about 1700 episodes. This might infer that the system built in the SAC algorithm with the use of an entropy term highly motivates to agent to continue to try interesting pathways allowing it to converge quickly. All DQN and SAC algorithms used an experience replay buffer of 100,000 size.

We used the following parameters in all agents:

- a learning rate l of 0.0005
- batch size of 64
- hidden layer size of 64
- gamma γ of 0.99

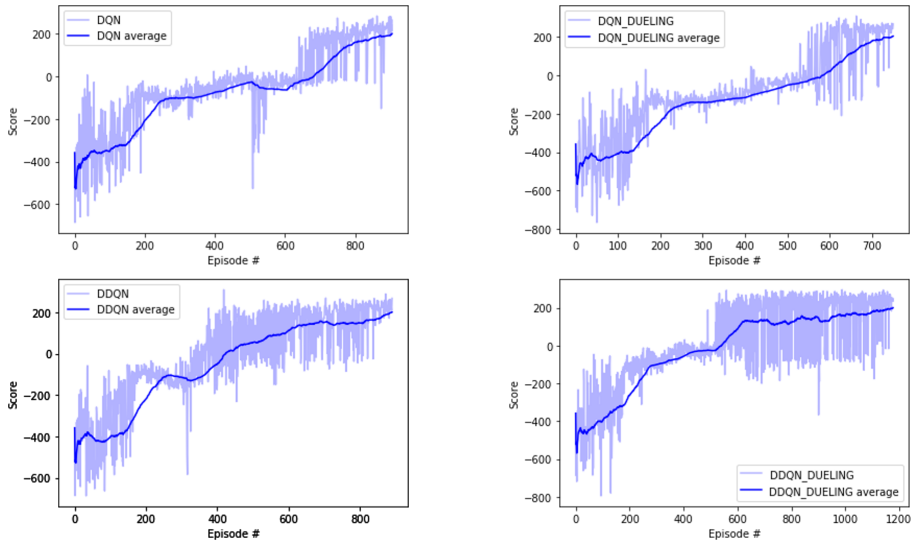

Figure 1: DQN plots

Figure 1. Each of the plots corresponds to a different variant of DQN, with the exact reward per epsiode and a moving average until environment is solved. Note that for the Double DQN the plot seems more stable as propsosed by the authors

3.1 DQN

The added hyper-parameters we have used in all of the DQN methods are: target model update every 4 iterations, and a interpolation parameter TAU of 0.001

According to our results, the double variant of the DQN family showed better performance than the rest, but in the case of adding the dueling network architecture, vanilla DQN showed better results than the double.

We have also tried different discretizations of the first action space, but its benefits weren't that significant. The second action space only has 3 different actions, so we included them in all our agents. All of our DQN agents were able to converge in a reasonable time in every run.

3.2 SAC and PPO

The SAC algorithm was trained using a similar parameters as the DQN, and with that seems to solve the environment in a much faster pace than any of the DQN variants (See Fig. 2). the only parameters that were changed were TAU and the update cycle, to 0.01 and every 2 steps respectively, as not alternating them cause what seems as a divergence or sub-optimal behavior of the agent.

We have also seen good results from changing the hidden layers size to 256 (instead of 64) neurons and the buffer size to 1 million. We omitted those runs as we wanted the comparison to be as similar as possible.

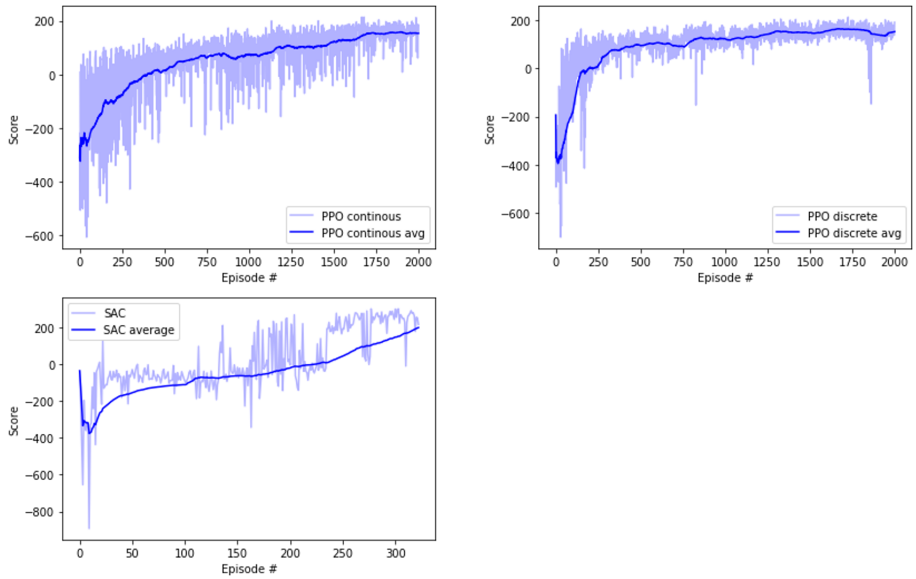

Figure 2: Policy methods plots

Figure 2. Each of the plots corresponds to a different algorithm as said in legend., with the exact reward per epsiode and a moving average until environment is solved. Note that for the PPO both runs are of agents that could accomplish to solve the environment after 2000 steps, although achieving about 150 average reward. one can see the fluctuation in those graphs.

The PPO method was trained with 2 more parameters: the ϵ we declared earlier of 0.2, and a K, the number of epochs we go over the current batch of examples to update our policy with. The PPO runs that were able to solve the Lunar Lander took 1700 steps on average. Other runs were fluctuating around a score of 150 for 2000 episodes, which then we decided to not continue with them, as shown in Fig. 2. Although one might assume from the plots that by continuing to train the agent might indeed converge to a solution, that wasn't the case for all runs.

3.3 Uncertainty

In real-world applications, some noise is usually present, therefore to accommodate to such environments we introduce a measure of randomness to the observation space as a zero-mean Gaussian noise with 0.05 standard deviation. We now theoretically evaluate the performance result of an agent on such environment.

As shown in [6] for parametric noise, we think that it is well shown that adding noise frequently boosts up performance. This can be explained by the intuition that when some uncertainty is present in the environment, we get unpredictable exploration as we are allowed to see new spaces we wouldn't see

otherwise.

We believe that due to the nature of the DQN algorithm and Policy methods, similar results should be seen in most techniques. The agents, although not directly build for noisy environments, are able to adapt and further learn different scenarios that affect their ongoing policy.

4 Conclusion and further discussion

In conclusion, this project of solving the Lunar Lander continuous environment using various reinforcement learning methods has showed some interesting insights. We found that the Deep Q-Learning methods were able to approximate the observation space well to solve the environment which too much effort, and some of their enhancements also showed contribution to the number of episodes needed to achieve solution while others stabilize the results. The policy methods are very promising, as we saw in our SAC experiments, which we believe the key to its success is the added entropy which allows a better exploration of the environment, avoiding sub-optimal behavior. We think these experiments has shown that there are multiple good solutions to a single task, and one should consider the environment features, such as how scarce getting samples is, the discrete or continuous nature of it, its complexity etc.

For future work, we are looking forward to understand whether different uncertainties might change the results we have seen, or even help them converge faster as seen in [6], by adding parameter noise to the neural network weights. We would also like to know how other well known algorithms like TD3[9], DDPG[3], amongst others perform on this environment. These tests can provide a fuller understanding both of the robustness of each methods, and a more general view on the suitability of each one.

5 Code

Our code is available at:

- https://colab.research.google.com/drive/1Xs9BxG81ZMQ2DGSkyBEdg5Vao3zh3mXx?usp=sharing

Resources that have been used during this work can be found at the Misc section down below.

6 Misc

1. Understanding Prioritized Experience Replay: https://danieltakeshi.github.io/2019/07/14/per/

2. Proximal Policy Optimization: https://spinningup.openai.com/en/latest/algorithms/ppo.html

3. PPO-PyTorch: https://github.com/nikhilbarhate99/PPO-PyTorch/
4. DQN-Atari-Agents: https://github.com/BY571/DQN-Atari-Agents
5. Policy-Gradient-Methods: https://github.com/cyoon1729/Policy-Gradient-Methods
6. How to implement Prioritized Experience Replay for a Deep Q-Network:ence.com/how-to-implement-prioritized-experience-replay-for-a-deep-q-network-a https://towardsdatasci710beecd77b
7. spinning-up https://github.com/openai/spinningup
8. In-depth review of Soft Actor-Critic https://towardsdatascience.com/in-depth-review-of-soft-actor-critic-91448aba63d4
9. Policy Gradient Algorithm https://lilianweng.github.io/lil-log/2018/04/08/policy-gradient-algorithms.html

References

[1] Watkins. "Q-Learning". In: *Springer* (June 1992).

[2] Volodymyr Mnih et al. "Playing Atari with Deep Reinforcement Learning". In: *arXiv e-prints*, arXiv:1312.5602 (Dec. 2013), arXiv:1312.5602. arXiv: 1312.5602 [cs.LG].

[3] Timothy P. Lillicrap et al. "Continuous control with deep reinforcement learning". In: *arXiv e-prints*, arXiv:1509.02971 (Sept. 2015), arXiv:1509.02971. arXiv: 1509.02971 [cs.LG].

[4] Hado van Hasselt, Arthur Guez, and David Silver. "Deep Reinforcement Learning with Double Q-learning". In: *arXiv e-prints*, arXiv:1509.06461 (Sept. 2015), arXiv:1509.06461. arXiv: 1509.06461 [cs.LG].

[5] Ziyu Wang et al. "Dueling Network Architectures for Deep Reinforcement Learning". In: *arXiv e-prints*, arXiv:1511.06581 (Nov. 2015), arXiv:1511.06581. arXiv: 1511.06581 [cs.LG].

[6] Matthias Plappert et al. "Parameter Space Noise for Exploration". In: *arXiv e-prints*, arXiv:1706.01905 (June 2017), arXiv:1706.01905. arXiv: 1706.01905 [cs.LG].

[7] John Schulman et al. "Proximal Policy Optimization Algorithms". In: *arXiv e-prints*, arXiv:1707.06347 (July 2017), arXiv:1707.06347. arXiv: 1707.06347 [cs.LG].

[8] Sutton Barto. "Reinforcement learning: An introduction". In: *MIT press* (May 2018).

[9] Scott Fujimoto, Herke van Hoof, and David Meger. "Addressing Function Approximation Error in Actor-Critic Methods". In: *arXiv e-prints*, arXiv:1802.09477 (Feb. 2018), arXiv:1802.09477. arXiv: 1802.09477 [cs.AI].

[10] Tuomas Haarnoja et al. "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor". In: *arXiv e-prints*, arXiv:1801.01290 (Jan. 2018), arXiv:1801.01290. arXiv: 1801.01290 [cs.LG].

[11] Hado van Hasselt et al. "Deep Reinforcement Learning and the Deadly Triad". In: *arXiv e-prints*, arXiv:1812.02648 (Dec. 2018), arXiv:1812.02648. arXiv: 1812.02648 [cs.AI].

[12] Soham Xin and Chengzhe. "Solving The Lunar Lander Problem under Uncertainty using Reinforcement Learning". In: *stanford journal* (June 2019).